College of Computing

Lab 3: SOLID Principles
Professor : Fahd KALLOUBI
Year : 2026/2027
Major : Computer Science (CS1)

In this Lab, we will put the five SOLID principles into practice. For each principle you will start from a small Java design that works but is hard to change, name the smell, and refactor it. The lab ends with a project: a small Java application that works but violates several principles at once, which you will review and refactor.

Learning outcomes:

- Detect SOLID violations in real code and name the design smell behind them
- Refactor a class toward SOLID using interfaces, polymorphism and constructor injection
- Explain how each refactoring limits the cost of the next change
- Judge when applying a principle is worth the added complexity

Work in one Java project, with one package per problem (`problem1`, `problem2`, …). For every problem, keep the original version in a sub-package (e.g., `problem1before`) and write your refactored version in a sub-package (e.g., `problem1after`), so that both can be compared.

Problem 1:

Single Responsibility Principle. Create the class `Invoice` below, together with a `Test` class that builds an invoice with three lines and calls its three public methods.

1. List the actors (people or teams) who could ask for a change in this class. How many reasons to change does `Invoice` have?
2. Accounting asks that `subtotal()` now applies a 5% discount above 1000 MAD. Which other outputs change silently? Which symptom is this?
3. Now, split the class by actor:
    a. `InvoiceData`: the customer and the lines, with no behaviour other than accessors (a record `InvoiceLine(String item, double price)` is welcome)
    b. `InvoiceCalculator`: `subtotal(InvoiceData)` and `total(InvoiceData)`
    c. `InvoicePrinter`: `print(InvoiceData)`, which returns the formatted text
    d. `InvoiceRepository`: `save(InvoiceData)`
4. Rewrite the `Test` class so that it produces the output below.
5. The heuristic says: ‘if you need “and” to describe a class, it probably does too much’. Describe each of your four classes in one sentence without using “and”.

Expected Output:

```
Invoice for ALAMI Samir
Laptop       9000.00
Mouse         150.00
USB cable      50.00
Subtotal     9200.00
TOTAL       11040.00
```

```
public class Invoice {
    private final String customer;
    private final List<String> items = new ArrayList<>();
    private final List<Double> prices = new ArrayList<>();

    public Invoice(String customer) { this.customer = customer; }

    public void addLine(String item, double price) {
        items.add(item);
        prices.add(price);
    }
    private double subtotal() {
        double sum = 0;
        for (double p : prices) sum += p;
        return sum;
    }
    public double total() {                        // 20% VAT
        return subtotal() * 1.20;
    }
    public String print() {
        StringBuilder sb = new StringBuilder();
        sb.append("Invoice for ").append(customer).append("\n");
        for (int i = 0; i < items.size(); i++)
            sb.append(line(items.get(i), prices.get(i)));
        sb.append(line("Subtotal", subtotal()));
        sb.append(line("TOTAL", total()));
        return sb.toString();
    }
    private String line(String label, double value) {
        return String.format("%-12s %8.2f%n", label, value);
    }
    public void save() throws IOException {
        try (FileWriter fw = new FileWriter(customer + ".txt")) {
            fw.write(print());
        }
    }
}
```

Problem 2:

Open / Closed Principle. A delivery company computes the shipping cost of a parcel with the following class.

```
public class ShippingCalculator {
    public double cost(String carrier, double weightKg) {
        if (carrier.equals("STANDARD")) {
            return 20 + 5 * weightKg;
        } else if (carrier.equals("EXPRESS")) {
            return 50 + 8 * weightKg;
        } else if (carrier.equals("DRONE")) {
            if (weightKg > 2) throw new IllegalArgumentException("Too
heavy");
            return 80;
        }
        throw new IllegalArgumentException("Unknown carrier: " + carrier);
    }
}
```

1. The company adds a “PICKUP” option (free under 5 kg, 10 MAD otherwise). re-test and redeploy with the current design.
2. In `problem2after` subpackage, declare an interface `ShippingMethod` with the methods `String name()` and `double cost(double weightKg)`.

3. Create one class per carrier: `StandardShipping`, `ExpressShipping`, `DroneShipping`.
4. Write a class `Parcel` with a weight and a method `double shippingCost(ShippingMethod method)`. `Parcel` must contain no `if` and no `switch` on the carrier.
5. Now add `PickupShipping`. Check that you did not open any existing file. (only Test changes, to add the new carrier to the list)
6. In the `Test` class, put the four methods in a `List<ShippingMethod>` and print the cost of a 2 kg parcel for each one.
7. Which design pattern did you just implement?
8. Next year, the cost will also depend on the destination country. Is your design closed against this change? What does this tell you about "closing against every change"?

Expected Output:

```
Standard: 30.00 MAD
Express: 66.00 MAD
Drone: 80.00 MAD
Pickup: 0.00 MAD
```

Problem 3:

Liskov Substitution Principle. A bank charges a monthly fee of 10 MAD on every account (don't forget to create subpackage `problem3before`).

```
public class Account {
    protected double balance;
    public Account(double balance) { this.balance = balance; }

    public void deposit(double amount) { balance += amount; }

    public void withdraw(double amount) {
        if (amount > balance)
            throw new IllegalArgumentException("Insufficient funds");
        balance -= amount;
    }
    public double getBalance() { return balance; }
}

public class FixedTermAccount extends Account {
    public FixedTermAccount(double balance) { super(balance); }

    @Override
    public void withdraw(double amount) {
        throw new UnsupportedOperationException("Locked until maturity");
    }
}

public class Bank {
    public static void chargeMonthlyFee(List<Account> accounts) {
        for (Account a : accounts) a.withdraw(10);
    }
}
```

1. In a `Test` class, create an `Account` with 500 MAD and a `FixedTermAccount` with 1000 MAD, put both in a list and call `Bank.chargeMonthlyFee`. What happens?

2. “A fixed-term account IS-A bank account”. Why is this sentence true in the real world but false for these two classes?
3. Which of the four rules of substitutability (preconditions, postconditions, invariants, exceptions) does `FixedTermAccount` break?
4. A colleague fixes the crash by adding `if (a instanceof FixedTermAccount) continue;` in `chargeMonthlyFee`. Which other principle is now violated, and what happens when a third kind of account appears?
5. In the package `problem3after`, rethink the hierarchy:
    a. an interface `Account` with `deposit(double)` and `getBalance()`
    b. an interface `WithdrawableAccount` that extends `Account` and adds `withdraw(double)`
    c. `CurrentAccount` implements `WithdrawableAccount`; `FixedTermAccount` implements only `Account`
6. Change `chargeMonthlyFee` so that the compiler, and not a runtime exception, prevents charging a fixed-term account.
7. Add a class `StudentAccount` that extends `CurrentAccount` and overrides `withdraw` so that it refuses any amount above 200 MAD per operation. Is `StudentAccount` substitutable for `CurrentAccount`? Justify your answer with the precondition rule, then propose a design that keeps the limit without breaking the contract.

Expected Output (question 6):

```
Current account balance: 490.0
Fixed-term account balance: 1000.0
```

Problem 4:

Interface Segregation Principle. The university information system describes every person with a single interface.

1. Implement `Professor` on the same interface. How many of its methods throw or do nothing? Count them for `Student` as well.
2. The method `gradeExam` receives a new parameter `int examId`. Which classes must be edited and recompiled, although they never grade anything?
3. In package `problem4after`, replace the fat interface with role interfaces designed from the point of view of their clients: `Learner` (enroll, pay tuition), `Teacher` (teach, grade) and `Researcher` (publish).
4. Implement `Student` (a learner), `Professor` (a teacher and a researcher) and `TeachingAssistant` (a learner and a teacher). No method may throw `UnsupportedOperationException` or have an empty body.
5. Write two clients, each depending on one role only:
    a. `Registrar` with `void enrollAll(List<Learner> learners, String code)`
    b. `ExamBoard` with `void collectGrades(List<Teacher> teachers, String code)`
6. In the `Test` class, pass the same `TeachingAssistant` object to both clients.
7. Would five interfaces with one method each be a better design? Justify your answer.

Expected Output:

```
SAFI Amal enrolls in SE101
ALAMI Samir enrolls in SE101
Prof. KALLOUBI grades SE101
ALAMI Samir grades SE101
```

```java
public interface UniversityMember {
    void enrollInCourse(String code);
    void payTuition(double amount);
    void teach(String code);
    void gradeExam(String code);
    void publishPaper(String title);
}

public class Student implements UniversityMember {
    private final String name;
    public Student(String name) { this.name = name; }

    public void enrollInCourse(String code) {
        System.out.println(name + " enrolls in " + code);
    }
    public void payTuition(double amount) {
        System.out.println(name + " pays " + amount + " MAD");
    }
    public void teach(String code) {
        throw new UnsupportedOperationException();
    }
    public void gradeExam(String code) {
        throw new UnsupportedOperationException();
    }
    public void publishPaper(String title) {
        throw new UnsupportedOperationException();
    }
}
```

Problem 5:

Dependency Inversion Principle. The enrollment rule of the university is implemented as follows. `CsvStudentStore` appends one line per student to a CSV file and `EmailSender` simply prints the message it would send; write these two small classes yourself.

```java
public class EnrollmentService {                    // high-level policy
    private final CsvStudentStore store =
            new CsvStudentStore("students.csv");
    private final EmailSender sender =
            new EmailSender("smtp.example.com");

    public void enroll(String cne, String name, String email) {
        if (cne == null || cne.isBlank())
            throw new IllegalArgumentException("CNE is required");
        if (store.exists(cne))
            throw new IllegalStateException("Already enrolled: " + cne);
        store.append(cne, name, email);
        sender.send(email, "Welcome " + name + ", you are enrolled.");
    }
}
```

1. Draw the dependency arrows between the three classes. Does the arrow go from policy to detail, or from detail to policy?

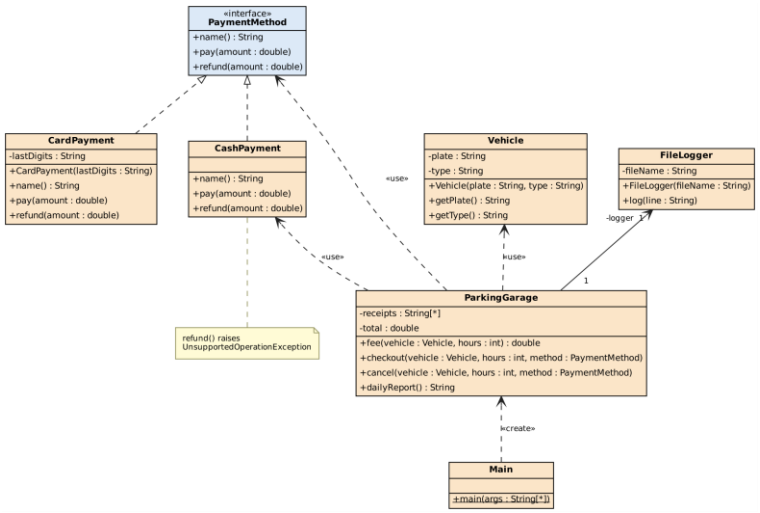
2. In `problem5after` package, declare two abstractions: `StudentRepository` (`boolean exists(String cne)`, `void save(String cne, String name, String email)`) and `Notifier` (`void notify(String to, String message)`). In which package should they live: next to `EnrollmentService` or next to the CSV and e-mail classes? Why?
3. Rewrite `EnrollmentService` so that it receives both abstractions through its constructor. The keyword `new` must no longer appear in this class.
4. Make `CsvStudentRepository` and `EmailNotifier` implement the abstractions.
5. Add an `SmsNotifier` and use it in the `main` method. Check that `EnrollmentService` was not modified.
6. Explain in two sentences the difference between dependency inversion (DIP) and dependency injection (DI). Which one did you apply in question 3, and which one in question 4?

Expected Output (question 7):

```
[SMS to 0612345678] Welcome SAFI Amal, you are enrolled.
Already enrolled: 22885676
```

Project: Refactoring a parking garage application

Fork and clone the following project. It contains a Java application that manages the payments of a parking garage (package `parking`, seven classes). It computes the fee of a vehicle, takes the payment by card or in cash, cancels a payment, prints a daily report and logs every receipt in the file `parking.log`. The application compiles and runs, but every new requirement forces the team to edit the class `ParkingGarage`. The class diagram below shows its current design; it is also available in the folder `docs` of the project.



Output of the current version:

```
[CARD ****4242] paid 15.0 MAD
[CASH] received 4.0 MAD
[CARD ****4242] paid 60.0 MAD
Refund 4.0 MAD at the desk for 67890-B-1
[CARD ****4242] refunded 60.0 MAD
=== Daily report ===
12345-A-6 | CAR | 3 h | 15.0 MAD | CARD
67890-B-1 | MOTORBIKE | 2 h | 4.0 MAD | CASH
24680-D-8 | TRUCK | 4 h | 60.0 MAD | CARD
Total: 15.0 MAD
```

1. Read the seven classes and compare them with the class diagram. How many actors (people or teams) can ask for a change in `ParkingGarage`? Name them.
2. Code review. Fill in a table with four columns (principle, class, line(s) of code, smell). You should find at least one violation of each of the five principles.
3. For each change request below, list the existing files that must be edited with the current design:
    a. a new vehicle type `BUS`, charged 15 MAD per hour
    b. a new payment method by mobile wallet, which can be refunded
    c. receipts logged in a database instead of a file
    d. a daily report in CSV format
4. In `ParkingGarage.cancel`, remove the test `method instanceof CashPayment` so that `method.refund(amount)` is always called, and run the program again. What happens? Which principle does `CashPayment` break, and why is the `instanceof` test not an acceptable fix?
5. Try to write a test that checks the fee of a truck parked for 4 hours and its checkout, without creating the file `parking.log`. Why is it impossible with the current design?
6. Draw the class diagram of a refactored design before writing any code. It should contain at least:
    a. an abstraction for the tariff of a vehicle, so that the `switch` on the type disappears
    b. a separate abstraction for the payment methods that can be refunded, so that no class throws `UnsupportedOperationException`
    c. a class in charge of formatting the receipts and the daily report
    d. an abstraction for the log, owned by the business logic
    e. an orchestrating class `ParkingService` that receives its collaborators through its constructor
7. Implement your design in the package `parking.refactored`, one principle at a time, and run `Main` after each step. The refactored program must print exactly the same output as the original one.
8. With your design, what happens at compile time if a developer tries to cancel a cash payment through `ParkingService`? Compare with question 4.
9. Write a class `ParkingServiceTest` with a `main` method that checks the fees of the three vehicles and the total of the daily report without creating any file. Which test double did you need?
10. Validate your design with the four change requests of question 3. For each one, write down the existing files you had to open. The goal is: none, except `Main`.

Deliverables: the refactored project, the class diagram of question 6 and the review table of question 2.